Streaming: delivering data incrementally over time

> ***How to read this file:*** *the first half is GENERAL streaming knowledge (transferable to any system/job/interview). The second half (marked "WORKED EXAMPLE") is how the WCX repo instantiates it. Learn the general model; use the repo to make it concrete.* ***Trust status:*** *repo specifics verified against live code 2026-07-28; re-grep symbols before citing line numbers. General concepts are stable industry knowledge.* ***Related:*** *[[kb-concurrency-async]] (generators/yield, non-blocking IO that make streaming work), [[kb-distributed-systems-patterns]] (idempotency, statelessness), [[kb-agent-loop]] (what produces the stream), [[kb-frontend-architecture]] (browser-side consumption). [[kb-harness-engineering]] (the capstone — streaming is how the harness delivers L1 output).*

PART 0 — WHY THIS MATTERS (philosophy & real-world connection)

Deep principle in one line: streaming trades a single big result for a sequence of small ones so the consumer can start before the producer finishes — *perceived* latency, not total latency, is what users feel.

Why it exists / what problem it solves. Two forces, one human and one physical. The human one: a first token in 200ms feels alive; a blank spinner for 20s feels broken — even if the *total* time is identical, your brain scores them completely differently. Streaming spends nothing on total latency and buys enormous perceived responsiveness. The physical one: some results are large or literally unbounded (a log tail, a live feed, an LLM that generates token-by-token), and you can't buffer gigabytes — or an infinite sequence — in memory to hand over "all at once." AI chat is what forced streaming from a niche technique into a mainstream default: once ChatGPT trained everyone to expect words appearing as they're written, a non-streaming chat UI feels obviously broken, so every serious LLM product now streams.

Real-world connection beyond this repo. You meet streaming everywhere once you look: every AI chat UI (ChatGPT, Claude) types at you; live dashboards and stock tickers push updates; `tail -f` and log viewers stream lines; video (Netflix/YouTube) streams as *segments*, not one file; collaborative editors stream keystrokes/ops; notification systems push events. At the data-engineering scale, "streaming data" (Kafka, event streams) is a *cousin* concept — same mental model of an unbounded appendable sequence, different machinery. And the SSE-vs-WebSocket choice is a recurring system-design interview question: one-way server→client → reach for SSE; client also needs to push mid-stream → WebSocket.

What breaks without understanding it (the failure modes bite you in exactly these spots):

- **Proxy buffering silently un-streams you** — nginx collects your chunks and releases them in a batch, so the user sees the whole answer appear at once. Everything "works" and nothing streams (§1.5).
- **Forgetting the framing buffer** → you treat network chunks as if they were logical messages and get corrupted or dropped frames (bytes ≠ messages; §1.4a).
- **No heartbeat** → a dead half-open connection is indistinguishable from a slow one, and its silence reads as "end of turn" (§1.4b).
- **Blocking the event loop / not using generators** → you materialize the whole result before sending, which means no streaming at all (§1.6).
- **Blind-retrying a send** whose response was lost → duplicates, because the write may have landed even though the ack didn't (§1.8).

The transferable mental model. A "stream" = a sequence of appendable chunks over time. Crucially, the *source* can be true-push OR polled-snapshots-diffed and the consumer can't tell the difference — that's the whole trick this repo leans on. On the wire it's always: **producer = a generator** (hands out one chunk, pauses, resumes) and **consumer = an accumulator with a framing buffer** (collects bytes, cuts complete frames on a delimiter, appends). Hold those two roles and their contract in your head and every streaming system reduces to the same shape.

PART 1 — GENERAL KNOWLEDGE

1.1 What "streaming" actually means

Streaming = delivering a response **incrementally over time** instead of all at once. The defining property: the consumer can start processing **before the producer is finished**. This matters whenever (a) the full result is large, (b) the full result is slow to produce, or (c) the result is unbounded (never "finished"). LLM chat hits all three — the answer is generated token-by-token over seconds.

Contrast with the default **request/response** model: client asks, server computes the *entire* answer, sends it in one body, closes. Simple, but the user stares at a spinner until the last byte. Streaming trades that for perceived latency: first token in ~200ms vs. full answer in 20s.

1.2 The family of "real-time" transports (know the whole map)

People conflate these; keep them distinct by **direction** and **framing**:

Tech	Direction	Transport	Typical use	Notes
Plain HTTP	1 req → 1 full resp	TCP	REST APIs	no streaming
HTTP chunked transfer	server → client, streamed	TCP/HTTP/1.1	file downloads, early streaming	the low-level mechanism SSE rides on
Server-Sent Events (SSE)	server → client only	HTTP	AI chat, live feeds, notifications	text frames, auto-reconnect, simple
WebSocket	full duplex (both push)	`ws://` upgrade from HTTP	multiplayer, collab editing, chat where client also pushes	binary or text, more complex, stateful conn
Long-polling	client re-asks, server holds	HTTP	fallback when SSE/WS blocked	client re-issues a request that the server holds open until data exists
Short-polling	client re-asks on a timer	HTTP	simple status checks	wasteful; fixed interval
gRPC streaming	uni or bidi	HTTP/2	service-to-service	binary Protobuf, multiplexed
WebRTC	peer-to-peer	UDP/SRTP	video/voice calls	not client-server
HTTP video (HLS/DASH)	server → client	HTTP segments	Netflix/YouTube	*segmented files*, usually NOT sockets

Choosing: need the client to push mid-stream? → WebSocket. One-way server→client? → SSE (simpler, auto-reconnects, works through most proxies). Can't hold a connection at all? → long-polling. Service-to-service, high throughput? → gRPC.

1.3 Server-Sent Events (SSE) in depth

SSE is **not a separate protocol** — it's a convention on top of ordinary HTTP with `Content-Type: text/event-stream`. The wire format is dead simple text frames separated by a blank line:

```
data: {"token": "Hello"}\n\n
data: {"token": " world"}\n\n
: this is a comment/heartbeat\n\n
```

- Each frame is `field: value` lines ending in a **blank line** (`\n\n` = the frame delimiter). Fields: `data:` , `event:` , `id:` , `retry:` .
- Lines starting with `:` are comments — used as **heartbeats** to keep the connection alive.
- The browser's built-in `EventSource` API parses this automatically and auto-reconnects (using the last `id:`). *Caveat:* many apps (including WCX) DON'T use `EventSource` because it only supports GET and no custom headers — they use `fetch()` + a manual reader loop instead, and hand-parse the `\n\n` frames.

Why SSE fits AI chat: the answer flows one way (server→browser); the client asks once and never pushes mid-answer. SSE gives you that with less machinery than WebSocket, and it survives most corporate proxies.

1.4 The two hard problems of streaming

(a) Framing / message boundaries. The network delivers *bytes in arbitrary chunks* that do NOT align with your logical messages. One TCP read might give you 1.5 SSE frames; the next gives the other 0.5 plus 2 more. **Every streaming consumer needs a buffer**: accumulate raw bytes, cut out *complete* frames on the delimiter, leave the partial remainder for next time. This is universal — it appears in SSE parsers, protocol decoders, line-readers, everything. Also: multi-byte UTF-8 characters can split across chunk boundaries — a stateful decoder (`TextDecoder(..., {stream:true})`) must hold the partial bytes.

(b) Liveness / dead connections. A held-open connection can die silently (half-open TCP: one side closed, the other never noticed). Without traffic you can't tell "slow but alive" from "dead." Solution: **heartbeats** — periodic no-op frames (SSE comment lines) from the server, and a **silence timeout** on the client (no frame for N seconds → assume dead → reconnect/abort). Intermediaries (nginx, load balancers) also have idle timeouts that will drop a quiet connection, so heartbeats keep it warm.

1.5 Backpressure & buffering (the subtle killers)

- **Buffering by intermediaries:** a reverse proxy (nginx) may *buffer* your stream — collecting chunks and releasing them in a batch — which silently defeats streaming (the user sees the answer appear all at once). You must tell the proxy not to buffer (`X-Accel-Buffering: no` for nginx) and flush each chunk.
- **Backpressure:** if the producer is faster than the consumer/network, data queues up. TCP provides flow control automatically; higher-level generator-based streaming (below) gets it for free because the generator only advances when the framework pulls the next value.

1.6 Generators & lazy production (the language mechanism)

Streaming on the producer side almost always uses **lazy iteration**. In Python that's a **generator** (a function with `yield`):

- `yield` is like `return` but **pauses** the function — freezing its local variables and position — and resumes from that exact spot when the next value is requested.
- This is what makes streaming possible: without it, `stream()` would have to build the *entire* answer in memory and return it all at once. With `yield`, it hands out one frame the instant it's ready, pauses, lets the framework flush it, then resumes.
- General pattern name: **lazy evaluation / pull-based iteration**. JS has `function*` / `yield` and async iterators; C# has `yield return` / `IEnumerable`; Go uses channels; Rust has iterators. Same idea everywhere: produce on demand, don't materialize the whole sequence.
- See [[kb-concurrency-async]] for how generators relate to coroutines and non-blocking IO.

1.7 Snapshot-polling vs. true push (a design axis)

Not all "streams" are push-based. Two ways to make data appear to stream:

- **True push:** the server sends each new piece as it's produced (SSE/WebSocket from the source).
- **Snapshot-polling + diffing:** the client repeatedly *pulls the full current state*, and computes what's NEW since last time, emitting only the delta. The consumer can't tell the difference — a "stream" is just *a sequence of appendable chunks over time*, regardless of whether the source pushed or was polled.

The diff trick (turning snapshots into a stream): keep a tiny bookmark of "how much I've already emitted" (e.g. a character count per message). Each poll returns the whole current text; emit only `text[already:]` (the new suffix) and advance the bookmark. This is memory-light (you store an int, not the content) and is exactly how you convert a stateful polled store into a token stream. (Git does the same conceptually: repeated diffs of a growing file = a stream of additions.)

1.8 Idempotency at the send boundary

When you POST a message then stream the reply, the POST's HTTP response can be **lost** (network blip) even though the write *landed* on the server. Blind retry → duplicate. The fix is **idempotency via verification**: before retrying, check whether your write actually appeared (e.g. "is there a new message matching my text?"). Only post if it genuinely didn't land. See [[kb-distributed-systems-patterns]] (idempotency, at-least/at-most-once delivery).

PART 2 — WORKED EXAMPLE: the WCX poll→diff→SSE→render pipeline

The WCX Engineering Copilot is a textbook instance of §1.7 (snapshot-polling faked into a stream) bridged to §1.3 (real SSE to the browser). Two connections, one contract:

```
SRE Agent  --(poll: get_messages, proactive REST snapshots)-->  backend handler  --(real SSE frames)--> browser
```

The backend is a **translator**: it POLLS the agent (request/response, §1.7) and PUSHES SSE to the browser (§1.3). `format_sse` builds `data:{...}\n\n` on the backend; the frontend's buffer splits on the same `\n\n` (§1.4a) — two ends of one pipe.

Send + receive = two separate HTTP calls, no readiness race

`post_message / create_thread` (send) and `get_messages` (receive) are distinct REST requests. There is NO push stream to "hook into." The agent WRITES its answer into the thread (durable storage — see [[kb-storage-model]]), not down a wire, so timing never causes a miss: slow agent → poll returns nothing-new → poll again; already-finished → poll returns it. **The thread IS the buffer.** Ordering guarantees correctness: (1) snapshot `baseline_ids`, (2) post question, (3) THEN poll — even a 5ms answer is found sitting in the thread. This is why polling-a-durable-store has no "am I ready to receive?" problem, unlike true push.

Polling backbone + optional SignalR hub

- **POLLING = mandatory backbone** — always runs; carries tool cards/reasoning/structure; complete fallback; streams text when the hub is off. Cannot be removed.
- **HUB (SignalR, a WebSocket §1.2) = optional accelerator** — fast text tokens + an authoritative `SignalProcessingComplete` signal. System works fully without it (laggier text + heuristic completion). Polling is source of truth, hub is turbo.
- When hub text is on, REST text is suppressed (`_suppress_rest_text`) to avoid double-emit.
- Hub "on" = THREE deciders ANDed (`use_hub_text = use_signalr and hub_text_enabled and not self.resume` , sre_stream.py:872): (1) NETWORK — did the WebSocket connect; (2) OPS CONFIG — `get_hub_text_streaming()` feature flag (no redeploy); (3) REQUEST TYPE — `not resume` (a hub is a LIVE channel with no memory of past pushes, so a post-reload resume falls back to REST whose first poll re-emits the full current answer). Any false → polling streams text. [general lesson: a live push channel can't replay history — that's why a durable pull store is the reliable backbone.]
- Timeouts (§1.4b): `_POLL_INTERVAL_SECONDS=0.8` normally, `_POST_COMPLETE_POLL_SECONDS=0.3` once completion seen; `_MAX_STREAM_SECONDS=600` cap; `_FIRST_RESPONSE_TIMEOUT=150` ; heartbeat when `time.monotonic() - self._last_emit >= _HEARTBEAT_INTERVAL_SECONDS(10)` → yield `_SSE_HEARTBEAT (": heartbeat\n\n")` . Frontend mirror: 90s silence → abort. [verified 2026-07-28: sre_stream.py:67,73,113,114,125-126,926]

The diff engine (§1.7 made concrete)

The backend is memory-light — it holds bookmarks, NOT the growing answer:

- `_emitted_text_len {msg_id → chars_sent}` (an int per message) — for TEXT.
- `_emitted_tool_payloads {msg_id → fingerprint tuple}` (status/name/output/args...) — for TOOLS.

```
already = self._emitted_text_len.get(mid, 0)   # get→0 default avoids KeyError on first sight
if len(text) > already:
    delta = text[already:]                      # emit ONLY the new suffix
    self._emitted_text_len[mid] = len(text)     # advance the high-water mark
    yield self._emit(StreamingMessage(content=delta, messageId=mid))
```

Walkthrough: poll1 "The error"(0→9 emit "The error"); poll2 "The error rate is"(9→17 emit " rate is"); poll4 unchanged (22>22 false → emit NOTHING = dedupe). Tools re-emit only when the fingerprint CHANGES. [verified 2026-07-28: sre_stream.py:772-778]

`_process_messages` gets SNAPSHOTS, not new-only — the "not adding up" crux. Input = the tail snapshot = full current state of recent msgs INCLUDING already-emitted ones; the SAME id recurs across polls, longer each time (NOT duplicate messages in one list). Two filters: `baseline_ids` (drop pre-turn msgs) + the emitted trackers (drop already-emitted PARTS).

baseline_ids — two jobs (both = "new since baseline")

- **Streaming filter:** `if mid in baseline_ids: continue` → not-in-baseline = this turn's content (maps response→question positionally).

- **Send idempotency (§1.8):** on a lost `post_message` response, `_send_follow_up` verifies a new-since-baseline user msg matching text landed instead of blind-retrying → never duplicates a turn.

Tail paging (constant per-poll cost regardless of thread length)

- `tail_skip = max(0, self.baseline_message_count - _TAIL_SKIP_MARGIN)` (sre_stream.py:892; margin=3) = a positional OFFSET, sent as `skip=N` — NOT an id. `baseline_message_count` defaults to `len(baseline_ids)` (sre_stream.py:602). Skips immutable baseline → fetch only the tail.
- `get_messages(page_size=200, max_messages=2000, start_skip=0)` (sre_client.py:223-225): `page_size=200` = **pagination** per HTTP response (the data plane caps each response), not a per-poll new-msg cap. Loops `while len(all_messages) < max_messages`, sending `{skip, top:page_size, orderby:"timestamp"}`, until a short page ends it. [general: pagination = fetch a large list in bounded chunks; server-side sort + client stitch.]

How `_emit` actually sends — it DOESN'T; 4 layers (ties to §1.6)

1. `_emit → format_sse` = `f"data: {json.dumps(msg.to_payload())}\n\n"` — object → SSE STRING, sends nothing.
2. `yield self._emit(...)` — `stream()` is a generator (§1.6); hands out one frame, pauses.
3. `Response(generator, mimetype="text/event-stream")` — **Flask/WSGI iterates the generator and writes+flushes each chunk to the socket = THE ACTUAL SEND** (your code never touches the socket). Headers: `text/event-stream`, `Connection:keep-alive`, `Cache-Control:no-cache`, and CRITICAL `X-Accel-Buffering:no` (§1.5 — stop nginx batching, else the answer appears all-at-once, not typing). [verified 2026-07-28: handler.py:236-239, models.py:111-115]
4. OS TCP socket → bytes over the held-open connection → client `reader.read()` receives them. (The old Foundry path had the same shape but the streamer was an ADAPTER translating Foundry EVENTS → SSE frames — two vocabularies.)

Frontend receive → accumulate → render (§1.4a + §1.7 consumer side)

SSE read loop (`useAgentChat.ts`): `response.body.getReader()` → `await reader.read()` blocks for the next byte-block → `TextDecoder("utf-8").decode(value,{stream:true})` (holds partial multi-byte chars, §1.4a; decoder reused once) → append to `buffer` → cut complete frames on `\n\n` → strip `data:`, skip `:` /blank/ `[DONE]` → `JSON.parse` → `applyStreamChunk`.

Frontend does NOT compute deltas — the backend already sent them. `parsed.content` IS the fragment; the frontend just APPENDS (`appendTextDelta`, not `computeDelta`). Frontend = ACCUMULATOR, backend = PRODUCER. This is the browser-side of the ONE delta the backend produced from snapshots (§1.7) — two ends of one pipe.

agentState = live scratchpad for the ONE reply streaming now. `assistantSegments` is the master record; the rest are projections: `assistantContent` (text-only, rebuilt via `textFromSegments`), `assistantTools`, `isToolInProgress`, `assistantMessage`.

Segments (not one string): the answer interleaves text + tool calls, so it's an ordered `IMessageSegment[]` = `{type:"text",content,sourceMessageId}` | `{type:"tool",tool}` (a **discriminated union** — text OR tool, never both; enforced by construction + TypeScript). `appendTextDelta` grows the last segment IF text AND same `sourceMessageId`, else pushes new. Merging loses FRAME BOUNDARIES (deliberate — §1.4a network artifacts have no meaning) but NEVER order. `sourceMessageId` is the seam that keeps leaked chain-of-thought separable from answer text, so `reconcileReasoningTool` can swap it for a collapsed "Thought process" block. Two scales: `allMessages` grows by TURNS; `assistantSegments` by FRAGMENTS within one reply.

The typing effect = React re-rendering `state.messages` many times/sec (NO animation). Per text chunk: grow segment → `textFromSegments` rebuilds content → copy onto `assistantMessage` → `dispatch(UPDATE_LAST_ASSISTANT, {...assistantMessage})` (spread = new object ref) → reducer swaps the last assistant msg into a NEW array → `messages.map` re-runs → `<ReactMarkdown>` renders. `CopilotResponse` is `React.memo` (re-renders only when content or last text segment changed) — a perf guard so not every message re-renders per fragment. See [[kb-frontend-architecture]].

PART 3 — transferable takeaways (the interview/next-job version)

1. Streaming = process-before-complete; pick the transport by direction (SSE one-way, WebSocket duplex, long-poll fallback).
2. Every streaming consumer needs a **framing buffer** (bytes ≠ messages) and **liveness handling** (heartbeats + silence timeout).
3. Watch for **proxy buffering** silently un-streaming you (`X-Accel-Buffering:no`).
4. A "stream" can be faked from **snapshot-polling + a delta bookmark** — the consumer can't tell.
5. Producer side is usually a **generator** (`yield`) for lazy, backpressure-friendly production.
6. At the send boundary, get **idempotency** right (verify-then-retry, never blind-retry).